

SIMATS ENGINEERING

ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA1304

COURSE OUTCOME COVERED:

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL: 3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:** 100 **Duration:** 1 Hour

CLASS TEST

Code of Conduct

I **Yuvann Nithish R (192411267)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: YUVANN NITHISH R

Question Paper

Q1. Design a PDA to accept strings with 'n' occurrences of 'a' followed by 'n' occurrences of 'b', where 'n' is a non-negative integer.

$$L = \{a^n b^n \mid n \geq 0\}$$

Q2. Convert the given context-free grammar (CFG) into a Pushdown Automaton (PDA) that recognizes the same language.

$$E \rightarrow E+E \mid E^*E \mid (E) \mid I$$

$$I \rightarrow Ia \mid Ib \mid I0 \mid I1 \mid a \mid b$$

Q3. Design a TM to accept the strings with 'n' occurrences of 'a' followed by 'n' occurrences of 'b', where 'n' is a non-negative integer.

$$L = \{0^n 1^n \mid n \geq 1\}$$

Q4. Design a Pushdown Automaton for the language representing a's followed by b's, where the number of b's must be twice the number of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

Q5. Construct a Turing Machine to recognize the language. Write the logic used for the design and the formal definition.

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Solutions

Question 1 — PDA for $L = \{a^n b^n \mid n \geq 0\}$

Design Logic

A pushdown automaton (PDA) can accept this language because a stack can be used to remember exactly how many a's have been read. Every 'a' read while still in the 'a-reading' phase pushes a marker onto the stack; every 'b' read afterwards pops one marker. If the input finishes at exactly the moment the stack returns to its initial symbol Z_0 , the number of a's equalled the number of b's, so the string is accepted. Since $n \geq 0$, the empty string must also be accepted directly in the start state.

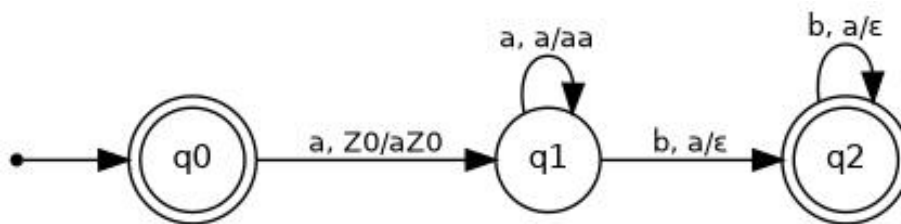
Formal Definition

$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{a, b\}$
- $\Gamma = \{a, Z_0\}$
- Start state: q_0 Initial stack symbol: Z_0
- $F = \{q_0, q_2\}$ (q_0 accepts ϵ ; q_2 accepts once every a has been matched)

Transition Function δ

State	Input	Stack Top	New State	Stack Action
q_0	a	Z_0	q_1	push a $\rightarrow aZ_0$
q_1	a	a	q_1	push a $\rightarrow aa$
q_1	b	a	q_2	pop a $\rightarrow \epsilon$
q_2	b	a	q_2	pop a $\rightarrow \epsilon$



Diagram

Trace — Acceptance of $w = aabb$ ($n = 2$)

An instantaneous description (ID) is written as (state, remaining input, stack contents), with the top of the stack on the left.

Step	Rule Applied	Instantaneous Description (state, input, stack)
0	—	$(q_0, aabb, Z_0)$
1	$\delta(q_0, a, Z_0) = (q_1, aZ_0)$	(q_1, abb, aZ_0)
2	$\delta(q_1, a, a) = (q_1, aa)$	(q_1, bb, aaZ_0)
3	$\delta(q_1, b, a) = (q_2, \epsilon)$	(q_2, b, aZ_0)
4	$\delta(q_2, b, a) = (q_2, \epsilon)$	$(q_2, \epsilon, Z_0) \rightarrow \text{ACCEPT } (q_2 \in F)$

Since the input is fully consumed and the machine halts in an accepting state, $aabb \in L$. Any string with an unequal number of a's and b's, or with a b before all a's are pushed, has no applicable transition and is rejected.

Question 2 — Converting the given CFG into an equivalent PDA**Design Logic**

Every context-free grammar $G = (V, T, P, S)$ can be converted into an equivalent single-state PDA that simulates left-most derivations of G on its stack. The stack initially holds the start symbol. At every step the PDA either (a) replaces a non-terminal on top of the stack by the right-hand side of one of its productions (an ϵ -move, representing one derivation step), or (b) matches a terminal on top of the stack against the next input symbol and pops it. The string is accepted when the input is exhausted and the stack becomes empty.

Standard Construction

For $G = (V, T, P, S)$, construct $P' = (\{q_0\}, T, V \cup T, \delta, q_0, S, \emptyset)$ accepting by empty stack, with:

- (1) $\delta(q_0, \epsilon, A) \ni (q_0, \alpha)$ for every production $A \rightarrow \alpha$ in P
- (2) $\delta(q_0, a, a) = (q_0, \epsilon)$ for every terminal $a \in T$

Applied to the Given Grammar

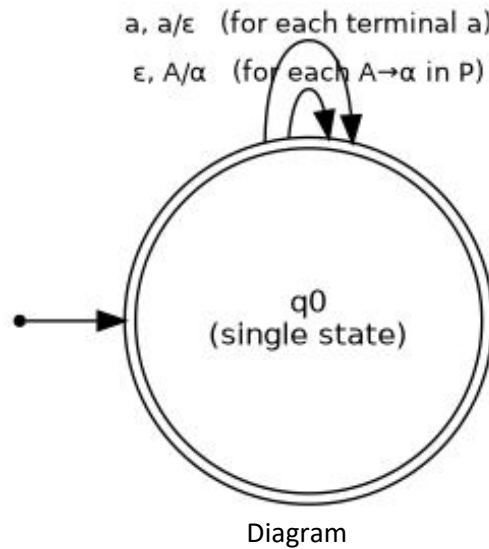
Non-terminals $V = \{E, I\}$ **Terminals** $T = \{+, *, (,), a, b, 0, 1\}$ **Start symbol** = E

Production (push) rules:

- $\delta(q_0, \epsilon, E) = \{(q_0, E+E), (q_0, E*E), (q_0, (E)), (q_0, I)\}$
- $\delta(q_0, \epsilon, I) = \{(q_0, Ia), (q_0, Ib), (q_0, I0), (q_0, I1), (q_0, a), (q_0, b)\}$

Terminal (match/pop) rules:

- $\delta(q_0, a, a) = (q_0, \epsilon)$ $\delta(q_0, b, b) = (q_0, \epsilon)$
- $\delta(q_0, +, +) = (q_0, \epsilon)$ $\delta(q_0, *, *) = (q_0, \epsilon)$
- $\delta(q_0, (, () = (q_0, \epsilon)$ $\delta(q_0,),)) = (q_0, \epsilon)$
- $\delta(q_0, 0, 0) = (q_0, \epsilon)$ $\delta(q_0, 1, 1) = (q_0, \epsilon)$



Worked Example — string $w = a+b$

Left-most derivation in G :

$E \Rightarrow E+E \Rightarrow I+E \Rightarrow a+E \Rightarrow a+I \Rightarrow a+b$

Corresponding PDA simulation (stack shown top $\rightarrow$ bottom; initial stack = E):

Remaining Input	Stack (top $\rightarrow$ bottom)	Move Used
$a+b$	E	start
$a+b$	$E + E$	$E \rightarrow E+E$
$a+b$	$I + E$	$E \rightarrow I$
$a+b$	$a + E$	$I \rightarrow a$
$+b$	$+ E$	match ' a '
b	E	match ' $+$ '
b	I	$E \rightarrow I$
b	b	$I \rightarrow b$
ϵ	ϵ	match ' b ' $\rightarrow$ ACCEPT (empty stack)

Because the stack empties exactly when the input is exhausted, the PDA accepts $a+b$ — confirming the construction correctly mirrors the grammar's left-most derivations for any string in $L(G)$.

Question 3 — Turing Machine for $L = \{0^n 1^n \mid n \geq 1\}$

Design Logic

A single-tape TM cannot 'count' the way a PDA does, so instead it repeatedly cancels one 0 against one 1: it finds the left-most unmarked 0, replaces it with X, scans right past all remaining 0's and already-matched Y's to find the left-most unmarked 1, and replaces that with Y. The head then returns to the left end and repeats. When no unmarked 0 remains, the machine scans right to confirm only Y's remain (no leftover, unmatched 1's) before accepting. If at any point an expected symbol is missing, the machine has no valid transition and rejects — this correctly rejects strings that are not of the form $0^n 1^n$.

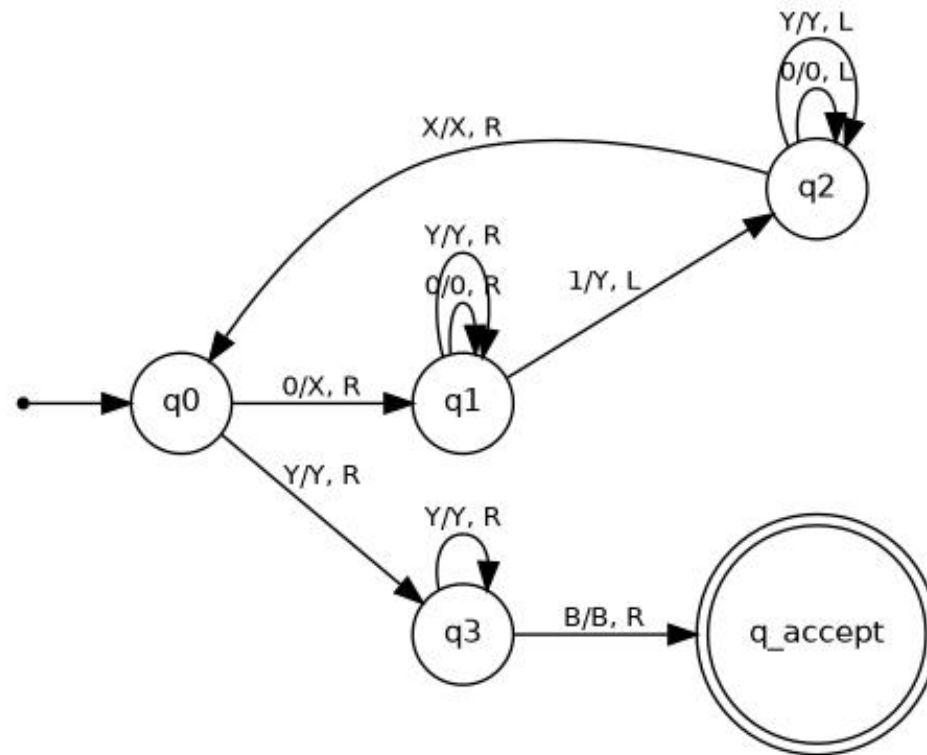
Formal Definition

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where

- $Q = \{q_0, q_1, q_2, q_3, q_{\text{accept}}\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, X, Y, B\}$ (B = blank)
- Start state: q_0 $F = \{q_{\text{accept}}\}$

Transition Function δ

State	Read	Write	Move	Next State	Purpose
q_0	0	X	R	q_1	mark a 0, go find its matching 1
q_0	Y	Y	R	q_3	no 0's left → verify the rest
q_1	0	0	R	q_1	skip remaining unmarked 0's
q_1	Y	Y	R	q_1	skip already-matched 1's
q_1	1	Y	L	q_2	mark the matching 1, turn back
q_2	0	0	L	q_2	return left over 0's
q_2	Y	Y	L	q_2	return left over Y's
q_2	X	X	R	q_0	reached the boundary → repeat
q_3	Y	Y	R	q_3	confirm all 1's were matched
q_3	B	B	R	q_{accept}	blank reached → ACCEPT

Figure 3 — State diagram of the TM for $L = \{0^n 1^n \mid n \geq 1\}$ **Trace (algorithm walk-through) for $w = 0011$**

Pass	Tape after the pass	Action
Start	0 0 1 1	head at left end, state q0
1	X 0 Y 1	left-most 0 $\rightarrow$ X; matching right-most-available 1 $\rightarrow$ Y; head returns
2	X X Y Y	next 0 $\rightarrow$ X; next 1 $\rightarrow$ Y; head returns
3	X X Y Y	state q0 sees Y (no 0's left) $\rightarrow$ switch to q3, scan right
4	X X Y Y (B)	q3 skips the Y's, reaches blank $\rightarrow$ q_accept

All symbols were consumed in matched X/Y pairs with none left over, so 0011 is accepted, correctly reflecting that it has an equal count of 0's and 1's with 0's preceding 1's.

Question 4 — PDA for $L = \{a^n b^{2n} \mid n \geq 1\}$ **Design Logic**

The number of b's must be exactly twice the number of a's. The stack can enforce this ratio directly: for every a read, push two stack symbols (instead of one); for every b read, pop exactly one symbol. If the input ends precisely when the stack has emptied back to Z0, the number of pops (= number of b's) equals twice the number of pushes-per-a (= number of a's), i.e. $|b| = 2 \cdot |a|$.

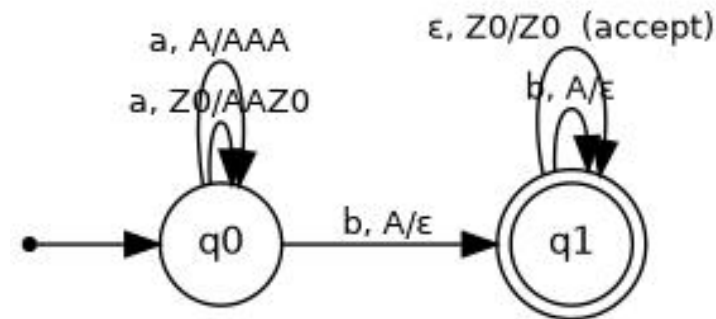
Formal Definition

$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where

- $Q = \{q_0, q_1\}$
- $\Sigma = \{a, b\}$
- $\Gamma = \{A, Z_0\}$
- Start state: q_0 Initial stack symbol: Z_0 $F = \{q_1\}$

Transition Function δ

State	Input	Stack Top	New State	Stack Action
q_0	a	Z_0	q_0	push AA $\rightarrow$ AAZ0
q_0	a	A	q_0	push AA $\rightarrow$ AAA (net +2 A's)
q_0	b	A	q_1	pop A $\rightarrow \epsilon$ (switch to popping phase)
q_1	b	A	q_1	pop A $\rightarrow \epsilon$
q_1	ϵ	Z_0	q_1	stack empty of A's $\rightarrow$ ACCEPT



Diagram

Trace — Acceptance of $w = aabbbb$ ($n = 2$)

Step	Rule Applied	Instantaneous Description (state, input, stack)
0	—	(q0, aabbbb, Z0)
1	$\delta(q0, a, Z0) = (q0, AAZ0)$	(q0, abbbb, AAZ0)
2	$\delta(q0, a, A) = (q0, AAA)$	(q0, bbbb, AAAAZ0)
3	$\delta(q0, b, A) = (q1, \epsilon)$	(q1, bbb, AAAZ0)
4	$\delta(q1, b, A) = (q1, \epsilon)$	(q1, bb, AAZ0)
5	$\delta(q1, b, A) = (q1, \epsilon)$	(q1, b, AZ0)
6	$\delta(q1, b, A) = (q1, \epsilon)$	(q1, ϵ , Z0) $\rightarrow$ ACCEPT ($q1 \in F$)

2 a's produced 4 A's on the stack, and exactly 4 b's were needed to empty it — confirming $|b| = 2 \cdot |a|$, as required.

Question 5 — Turing Machine for $L = \{a^n b^n c^n \mid n \geq 1\}$ **Design Logic**

This language cannot be generated by any context-free grammar or accepted by any PDA (it fails the CFL pumping lemma, since a single stack cannot track two independent counts simultaneously), but a Turing machine's ability to move freely in both directions and repeatedly rewrite the tape makes it solvable. The strategy generalises Question 3: repeatedly cancel one a, one b and one c per pass. On each pass the machine marks the left-most unmarked a as X, moves right to mark the corresponding left-most unmarked b as Y, continues right to mark the corresponding left-most unmarked c as Z, and then returns to the start. This repeats until no unmarked a's remain, after which the machine scans right to confirm that only Y's and Z's (no left-over b's or c's) remain before accepting.

Formal Definition

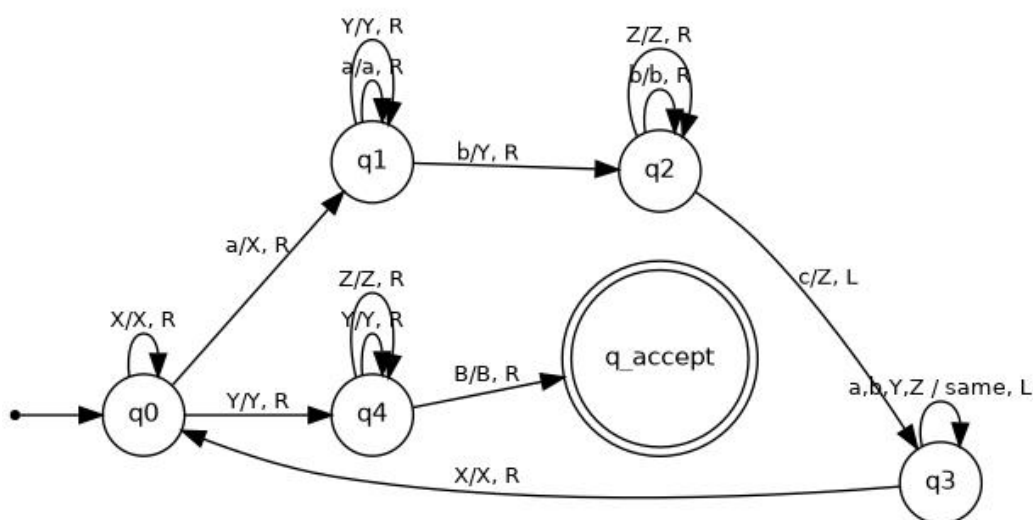
$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_{\text{accept}}\}$
- $\Sigma = \{a, b, c\}$
- $\Gamma = \{a, b, c, X, Y, Z, B\}$ ($B = \text{blank}$)
- Start state: q_0 $F = \{q_{\text{accept}}\}$

Transition Function δ

State	Read	Write	Move	Next State	Purpose
q0	X	X	R	q0	skip already-processed a's
q0	a	X	R	q1	mark next a, go find its b
q0	Y	Y	R	q4	no a's left $\rightarrow$ final check
q1	a	a	R	q1	skip remaining unmarked a's
q1	Y	Y	R	q1	skip already-matched b's

State	Read	Write	Move	Next State	Purpose
q1	b	Y	R	q2	mark the matching b, go find its c
q2	b	b	R	q2	skip remaining unmarked b's
q2	Z	Z	R	q2	skip already-matched c's
q2	c	Z	L	q3	mark the matching c, turn back
q3	a,b,Y,Z	same	L	q3	return left over the marked region
q3	X	X	R	q0	reached boundary → repeat
q4	Y	Y	R	q4	confirm all b's were matched
q4	Z	Z	R	q4	confirm all c's were matched
q4	B	B	R	q_accept	blank reached → ACCEPT



Diagram

Trace (algorithm walk-through) for $w = aabbcc$

Pass	Tape after the pass	Action
Start	a a b b c c	head at left end, state q0
1	X a Y b Z c	1st a→X, 1st unmarked b→Y, 1st unmarked c→Z; head returns
2	X X Y Y Z Z	2nd a→X, 2nd b→Y, 2nd c→Z; head returns
3	X X Y Y Z Z	q0 sees Y (no a's left) → switch to q4, scan right
4	X X Y Y Z Z (B)	q4 skips Y's and Z's, reaches blank → q_accept

Every a, b and c was cancelled in matched triples with nothing left over, so aabbcc is accepted — correctly verifying equal counts of a's, b's and c's in the order a...a b...b c...c.